xrepost — documentación completa

Herramienta para **repostear posts de usuarios concretos dentro de un rango de fechas** en X (Twitter), usando la sesión de Chrome que ya tienes abierta.

No usa la API de X ni pide contraseñas: controla tu navegador real mediante CDP (Chrome DevTools Protocol) a través del harness `browser-use`.

Índice

1. Qué es y qué hace
2. Estructura de la carpeta
3. Requisitos
   - 3.1 ¿Hace falta un modelo de IA?
4. Instalación (una sola vez)
5. Uso
6. Variables de entorno
7. Cómo funciona por dentro
8. Estados de salida y errores
9. Diagnóstico
10. Límites, riesgos y buenas prácticas
11. Cómo extenderlo
12. Verificación realizada
13. Preguntas frecuentes

---

1. Qué es y qué hace

`xrepost` responde a una pregunta concreta: *"¿qué publicó tal usuario entre el día A y el día B, y quiero repostearlo?"*

Hace cuatro cosas:

1. **Busca** los posts de una lista de usuarios en un rango de fechas, usando la búsqueda nativa de X (`from:`, `since:`, `until:`).
2. **Filtra** por fecha real, descarta respuestas y retweets, elimina duplicados, aplica filtros de palabras (`X_MATCH` / `X_EXCLUDE`) y muestra un listado con autor, fecha, texto y enlace; opcionalmente lo exporta a CSV o Markdown.
3. **Repostea** (opcional) cada uno de esos posts, con límite y pausas, verificando que el repost se hizo de verdad.
4. **Registra y deshace**: guarda cada repost en un historial para no repetirlo en ejecuciones futuras y permite revertirlo con `--undo-last` o `--undo`.

Por defecto **no repostea nada**: primero enseña la lista (modo *dry-run*). Solo actúa si se lo pides explícitamente con `X_APPLY=1`.

---

2. Estructura de la carpeta

```
~/Downloads/x-repost/
├── xrepost.sh                # interfaz: pregunta usuarios/fechas o recibe argumentos
├── xrepost.py                # lógica: búsqueda, filtrado, repost y verificación
├── README.md                 # este documento (texto, se actualiza primero)
└── xrepost-documentacion.pdf  # este mismo documento en PDF (13 páginas)
```

- **`xrepost.sh`** es un envoltorio Bash. Recoge los datos, los pasa como variables de entorno y ejecuta `browser-use < xrepost.py`.
- **`xrepost.py`** se ejecuta *dentro* del harness. No importa `browser-use`: las funciones (`goto_url`, `js`, `wait_for_load`, ...) ya están disponibles en el entorno de ejecución.

El historial y los estados no viven en la carpeta, sino en `~/.local/state/xrepost/history.jsonl` (una línea JSON por acción). Se puede cambiar con `X_HISTORY`; los `--csv` / `--md` se guardan donde indiques.

---

3. Requisitos

Pieza	Versión usada	Para qué
Google Chrome	153	El navegador que se controla
`browser-use` (CLI)	0.1.13	Harness que habla CDP
`browser-use` (paquete)	0.13.14	Librería
Python	3.12.14 (gestionado por `uv`)	Ejecuta el script
`uv`	0.12+	Instala y aísla Python/paquetes

Comprobar que todo está:

```
browser-use --version
browser-use --doctor
```

3.1 ¿Hace falta un modelo de IA?

No. `xrepost` no llama a ningún modelo de lenguaje en tiempo de ejecución. Es automatización determinista del DOM: órdenes fijas ("navega a esta URL", "lee este selector", "haz clic aquí") ejecutadas contra Chrome.

Hay dos cosas distintas que se llaman `browser-use`:

Componente	Qué es	¿Usa LLM?
CLI / harness (`browser-use`)	Controlador de Chrome por CDP. Expone `goto_url`, `js`, `wait_for_load`...	**No**
Librería `Agent` (Python)	Agente que decide los pasos por su cuenta	Sí, pero **no se usa aquí**

`xrepost.py` se ejecuta dentro del **harness**, no dentro de `Agent`. No hay ninguna decisión "inteligente" en el camino: la lista de acciones está escrita de antemano.

Verificación sobre el código instalado:

```
grep -rE "OPENAI_API_KEY|api.openai.com|api.anthropic.com" \
  ~/.local/share/uv/tools/browser-use/lib/python3.12/site-packages/browser_harness/
# → sin resultados
```

Las únicas llamadas de red del harness son:

Destino	Cuándo
`api.browser-use.com`	Solo si se usan navegadores en la nube (aquí no)
`eu.i.posthog.com`	Telemetría anónima; se apaga con `browser-use telemetry disable`

Es decir: el programa funciona sin ninguna clave de API de IA, sin conexión a un proveedor de modelos y sin coste por token. Con internet solo para llegar a X.

¿Dónde está entonces la IA? En el agente que construyó el script (OpenCode) mientras se conversaba con él. Una vez creado, el programa se lanza desde una terminal sin ningún modelo involucrado:

```
cd ~/Downloads/x-repost && ./xrepost.sh
```

Cuándo sí haría falta un modelo (opcional):

- Decidir *qué* posts merecen repost ("solo los de Kubernetes, no las ofertas de empleo").
- Resumir el timeline antes de aprobar.
- Redactar un comentario para un quote-repost.
- Sustituir el script por un agente que se adapte si X cambia el HTML.

Nada de eso es necesario para la función actual: usuarios + fecha → lista → repost verificado.

---

4. Instalación (una sola vez)

```
# 1. instalar el CLI y la librería en un entorno aislado con Python 3.12
uv tool install --python 3.12 browser-use

# 2. registrar la skill en OpenCode (opcional, para usarlo desde el agente)
browser-use skill install --target opencode --no-install
```

La skill queda en `~/.config/opencode/skills/browser-use/SKILL.md` y permite pedirle tareas al agente en lenguaje natural.

---

5. Uso

5.1 Arrancar Chrome con el puerto de control

Chrome tiene que estar escuchando en `9222`. Desde Chrome 136 el debugging remoto está **bloqueado en el perfil por defecto**, así que se usa un perfil dedicado:

```
google-chrome --user-data-dir="$HOME/.config/chrome-agent" \
              --remote-debugging-port=9222 &
```

En ese perfil, inicia sesión en X una vez (la sesión se guarda ahí).

Comprobar que el puerto responde:

```
curl -s http://127.0.0.1:9222/json/version | grep Browser
```

5.2 Modo interactivo (pregunta usuarios y fechas)

```
cd ~/Downloads/x-repost
./xrepost.sh
```

```
=== xrepost ===
Usuarios (separados por coma, @ opcional): kubernetesio, docker
Fecha desde (YYYY-MM-DD): 2026-10-27
Fecha hasta (Enter = la misma):
```

5.3 Modo con argumentos

```
./xrepost.sh <usuarios> <desde> [hasta]
```

Ejemplo	Qué hace
`./xrepost.sh kubernetesio 2026-10-27`	Lista los posts de ese día (dry-run)
`./xrepost.sh kubernetesio 2026-10-20 2026-10-28`	Lista el rango
`./xrepost.sh a,b,c 2026-10-27`	Varios usuarios a la vez
`X_MATCH="kubernetes,cloud" ./xrepost.sh a 2026-10-27`	Solo posts que contengan alguna de esas palabras
`X_EXCLUDE="empleo,oferta" ./xrepost.sh a 2026-10-27`	Descarta posts que contengan alguna
`X_APPLY=1 ./xrepost.sh kubernetesio 2026-10-27`	Repostea de verdad
`X_APPLY=1 X_LIMIT=3 X_DELAY=45 ./xrepost.sh a 2026-10-27`	Máximo 3, con 45 s entre cada uno

También se puede pasar todo por variables de entorno, sin argumentos:

```
X_USERS=kubernetesio X_FROM=2026-10-27 X_MATCH=cloud ./xrepost.sh
```

5.4 Exportar el listado

```
./xrepost.sh --csv posts.csv kubernetesio 2026-10-27
./xrepost.sh --md posts.md kubernetesio 2026-10-20 2026-10-28
```

El CSV lleva `fecha,autor,usuario,texto,url,estado`; el Markdown es una tabla con enlace al post. El estado es `solo listado` en dry-run y el resultado real (reposteado, ya estaba, fallo…) cuando se aplica. Se pueden usar ambos a la vez.

5.5 Historial

```
./xrepost.sh --history        # últimos 20 movimientos
./xrepost.sh --history 50     # últimos 50
```

Cada repost o undo se guarda en `~/.local/state/xrepost/history.jsonl` con fecha, ejecución, autor, handle, URL y resultado. Antes de repostear, la herramienta omite lo que ya consta como reposteado; para forzar que lo vuelva a procesar: `--force` (o `X_FORCE=1`).

5.6 Deshacer

```
./xrepost.sh --undo-last          # todo lo reposteado en la última ejecución
./xrepost.sh --undo-last 3        # los últimos 3 reposts
./xrepost.sh --undo https://x.com/usuario/status/1234567890
./xrepost.sh --undo 1234567890
```

Solo deshace reposts **hechos por la herramienta** (no los que ya estaban antes); verifica contra el DOM que el botón vuelve a ser `retweet`. Igual que al repostear, hay pausa `X_DELAY` entre cada uno.

5.7 Ayuda

```
./xrepost.sh --help
```

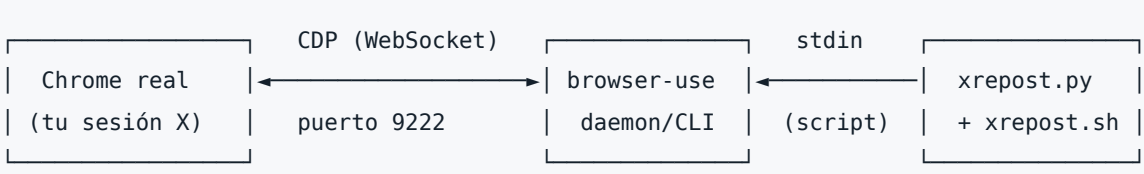
---

6. Variables de entorno

Variable	Por defecto	Significado
`X_USERS`	—	Lista separada por comas. El `@` es opcional
`X_FROM`	—	Fecha inicial `YYYY-MM-DD`
`X_TO`	igual a `X_FROM`	Fecha final **incluida**
`X_APPLY`	`0`	`0` = solo listar · `1` = repostear
`X_LIMIT`	`10`	Máximo de reposts en la ejecución
`X_DELAY`	`20`	Segundos de espera entre reposts
`X_SKIP_REPLIES`	`1`	`1` = ignorar respuestas · `0` = incluirlas
`X_SKIP_RETWEETS`	`1`	`1` = ignorar retweets (el autor original no es el buscado) · `0` = incluirlos
`X_MATCH`	—	Palabras separadas por comas; el post debe contener **alguna** (sin distinguir mayúsculas)
`X_EXCLUDE`	—	Palabras separadas por comas; si el post contiene **alguna**, se descarta
`X_FORCE`	`0`	`1` = ignorar el historial y volver a procesar lo ya reposteado
`X_HISTORY`	`~/.local/state/xrepost/history.jsonl`	Ruta del historial JSONL
`X_CSV` / `X_MD`	—	Exportación; normalmente se usan los flags `--csv` / `--md`
`BU_CDP_URL`	`http://127.0.0.1:9222`	Endpoint de Chrome

7. Cómo funciona por dentro

7.1 Arquitectura



- **Chrome con CDP**: una API para programar el navegador (navegar, evaluar JS, leer el DOM, hacer clic). Es la misma que usan las DevTools con F12.
- **browser-use**: mantiene un daemon conectado al navegador y expone funciones cortas que traducen a comandos CDP (`Page.navigate`, `Runtime.evaluate`, …). El daemon persiste entre

ejecuciones, por eso recuerda la pestaña actual.
- **xrepost.py**: decide *qué* hacer; no habla con Chrome directamente.

7.2 Recorrido de una ejecución

Paso 1 — el wrapper recoge los datos. `xrepost.sh` interpreta los flags (`--csv`, `--md`, `--force`, `--undo-last`, ...), toma usuarios y fechas de los argumentos (o de `X_USERS` / `X_FROM`, o los pregunta) y los exporta como variables de entorno. Después ejecuta:

```
exec browser-use < xrepost.py
```

El script entra por **stdin**; por eso no necesita importar nada del harness.

Paso 2 — construye la búsqueda. Traduce usuarios y fechas a la sintaxis de X:

```
https://x.com/search?q=from%3Akubernetesio%20since%3A2026-10-27%20until%3A2026-10-28&f=live
```

Parte	Por qué
`from:usuario`	limita al autor
`since:` / `until:`	rango de fechas; `until` es **exclusivo**, así que se suma un día
`f=live`	orden cronológico en vez del "Top" algorítmico

Paso 3 — navega y espera. `goto_url()` y `wait_for_load()`. Como X es una SPA, "cargado" no implica "con posts": `wait_articles()` sondea cada segundo hasta que aparece al menos un `article[data-testid="tweet"]` (máx. 12 s).

Paso 4 — extrae datos del DOM con JavaScript ejecutado dentro de la página:

Campo	Selector / fuente
`time`	atributo `datetime` del `<time>` (ISO real, no "2h")
`url`	`href` del enlace que contiene `/status/`
`user`	`[data-testid="User-Name"]` (nombre visible)
`handle`	segunda línea de `[data-testid="User-Name"]` (`@usuario`); si falta, se saca de la URL
`text`	`[data-testid="tweetText"]`
`ctx`	`[data-testid="socialContext"]` (contexto tipo "reposted", solo informativo)
`reply`	¿hay enlaces de perfil antes de `tweetText` ? → detecta respuestas por estructura, no por idioma

Paso 5 — filtra y pagina. Descarta lo que cae fuera del rango, descarta respuestas (`X_SKIP_REPLIES=1`, por estructura del DOM) y **retweets** (`X_SKIP_RETWEETS=1`, comparando el handle del autor con el usuario buscado), aplica `X_MATCH` / `X_EXCLUDE` sobre el texto, deduplica por URL de forma global (también entre usuarios de la misma ejecución) y hace `window.scrollBy(0, 1600)` hasta 8 veces para forzar el scroll infinito de X. En cada vuelta vuelve a extraer y acumula.

Paso 6 — decide y exporta. Consulta el historial y omite lo que ya consta como reposteado (salvo `--force`). Sin `X_APPLY=1` imprime el listado, escribe los `--csv` / `--md` si se pidieron y termina. Cero escrituras en X.

Paso 7 — repostea (solo con `X_APPLY=1`). Para cada URL:

1. `goto_url(post)` y espera.
2. Si ya existe `[data-testid="unretweet"]` → ya estaba reposteado, se salta.
3. Clic en `[data-testid="retweet"]` → X abre un menú.
4. Espera a que aparezca `[data-testid="retweetConfirm"]` ("Repost / Quote").
5. Clic en "Repost".
6. **Verifica**: si ahora existe `unretweet`, se hizo. Si no, lo reporta como fallo.
7. Anota el resultado en el historial y `sleep(X_DELAY)`; siguiente, hasta `X_LIMIT`.

Paso 8 — informa. Una línea por post con el resultado y el total reposteado. Si se pidió exportación, reescribe el CSV/Markdown con el estado final de cada post.

Paso 9 — deshacer (`--undo-last` / `--undo`). No busca ni repostea: lee el historial, elige los reposts hechos por la herramienta que siguen vigentes, entra en cada URL, pulsa `unretweet` (y confirma si X abre el menú) y verifica que el botón vuelve a ser `retweet`. Cada intento se apunta en el historial.

7.3 Decisiones de diseño (y por qué)

- **Clics por JS, no por coordenadas.** El harness ofrece `click_at_xy` (ratón real), pero en X el `scroll()` por rueda se colgaba (`Input.dispatchMouseEvent timed out`). Solución: `element.click()` desde la página y scroll con `window.scrollBy`. Funciona porque React de X escucha eventos nativos de clic.
- **Verificar contra el DOM, no contra "no dio error".** Un clic puede fallar en silencio; el cambio `retweet` → `unretweet` es la única prueba fiable.
- **Fechas del `<time>`, no del texto visible.** X muestra "8h" o "Oct 27", que engaña al cruzar días u husos horarios.
- **Buscar en vez de recorrer el perfil.** Recorrer `x.com/usuario` exige decenas de scrolls y mezcla reposts y posts fijados; la búsqueda `from:` da exactamente autor y rango.
- **Dry-run por defecto.** Evita el escenario "me equivoqué de usuario y reposteé 200 posts".
- **Deduplicación global por URL** antes de actuar: el conjunto `SEEN` es único para toda la ejecución, no por usuario, así que un mismo post encontrado en la búsqueda de dos cuentas (o repetido por el scroll) se lista y se repostea una sola vez.
- **Los retweets se descartan por defecto.** La búsqueda `from:usuario` también devuelve lo que ese usuario ha retuiteado, y ahí el autor real es otro. Repostear eso sería redundante y atribuiría mal el contenido, así que se compara el handle del autor (extraído de `User-Name`, con la URL como respaldo) con el usuario buscado: si no coinciden, es un retweet y se ignora. Antes se buscaba el texto `"X reposted"`, que solo existe con la interfaz en inglés. Para incluirlos: `X_SKIP_RETWEETS=0`.
- **Las respuestas también se detectan por estructura, no por idioma.** X dibuja la línea "Replying to" (o "Respondiendo a", "Réponse à"…) como enlaces de perfil situados antes del texto del post. Basta con encontrar un enlace `@usuario` fuera de `User-Name` y de `tweetText` que preceda a este último; la lista multilingüe de textos queda solo como respaldo.

- **Historial como fuente de verdad para no repetir.** Cada acción se guarda en JSONL (`reposted`, `already`, `undone`…). Antes de repostear se omiten los IDs cuyo último evento indica que siguen reposteados. Es idempotente entre ejecuciones y sobrevive a reinicios; `--force` lo ignora cuando haces cambios a mano.
- **Deshacer solo lo que hizo la herramienta.** `--undo-last` no toca los posts que ya estaban reposteados antes (`already`), para no revertir acciones ajenas. Si el repost se deshizo a mano, el historial puede quedar desfasado: usa `--force` para volver a procesarlo.
- **Filtros de texto simples, no regex.** `X_MATCH` / `X_EXCLUDE` son subcadenas separadas por comas, sin distinguir mayúsculas: predecible, sin sorpresas de expresiones regulares sobre el contenido.
- **Exportar antes y después.** El CSV/Markdown se escribe al listar (estado `solo listado` o `pendiente`) y se reescribe al terminar con el resultado real, para poder revisar o archivar la tanda.

---

8. Estados de salida y errores

Cada post recibe uno de estos estados:

Estado	Significado
`reposteado`	Se hizo y se verificó
`ya estaba reposteado`	Se detectó `unretweet` antes de actuar; no se tocó
`no encontré el botón de repost`	El post no cargó o no es reposteable (p. ej. borrado o protegido)
`el menú no se abrió`	El clic no desplegó el menú de repost
`no se pudo confirmar`	Se hizo clic pero no apareció `unretweet` (posible fallo de red o límite de X)

Al deshacer (`--undo-last` / `--undo`):

Estado	Significado
`deshecho`	Se quitó el repost y se verificó que el botón vuelve a `retweet`
`no estaba reposteado`	El post ya no tenía repost; no se tocó
`no se pudo deshacer`	Se intentó pero `unretweet` sigue apareciendo

Además, si el historial ya tenía un post como reposteado, el listado avisa `(N ya reposteados en ejecuciones anteriores; usa --force para incluirlos)` y no lo procesa.

Códigos de salida:

- `0` — terminó (haya reposteado o no).
- `1` — faltan usuarios o fecha (lo devuelve el wrapper).
- `2` — faltan `X_USERS` o `X_FROM` dentro del script.

---

9. Diagnóstico

```
# estado general: Chrome, daemon, conexiones
browser-use --doctor

# log del daemon (aquí aparecen los errores reales)
tail -f ~/.config/browser-harness/tmp/bu-default.log

# ¿responde Chrome?
curl -s http://127.0.0.1:9222/json/version

# ver la última captura que hizo el script
ls -l ~/.config/browser-harness/tmp/shot.png

# historial de reposts y deshacer
./xrepost.sh --history 20
ls -l ~/.local/state/xrepost/history.jsonl
```

Errores típicos:

Síntoma	Causa	Solución
`chrome-not-running`	Chrome cerrado o sin puerto	Arrancarlo con `--remote-debugging-port=9222`
`Input.dispatchMouseEvent timed out`	Scroll por rueda en X	Ya evitado: el script usa `window.scrollBy`
`Sin posts en ese rango`	No hay posts, o la búsqueda no cargó	Probar el rango en el navegador a mano
`no se pudo confirmar`	Rate limit o fallo de red	Bajar `X_LIMIT`, subir `X_DELAY`

10. Límites, riesgos y buenas prácticas

- **Esto automatiza acciones en tu cuenta.** Un fallo del script actúa sobre tu cuenta real, no sobre una copia.
- **Reglas de X:** el repost automatizado y masivo entra en sus políticas de automatización. Con pocos reposts, espaciados y sobre contenido propio o de cuentas que te autorizan, el riesgo es bajo; con cientos, la cuenta acaba limitada o suspendida.
- **Recomendaciones:** `X_LIMIT` entre 3 y 10, `X_DELAY` entre 20 y 60 s, y no ejecutarlo en bucle.
- **Sesión:** usa un perfil de Chrome dedicado (`~/.config/chrome-agent`), no tu perfil personal con banca y correo.
- **Volumen alto:** el camino correcto es la API oficial (`POST /2/users/:id/retweets`), no el navegador.
- **Credenciales:** el script nunca las toca; aprovecha la sesión ya iniciada.

- **El historial no es un freno de seguridad.** Evita repetir lo ya hecho, pero no limita el volumen: `X_LIMIT` y `X_DELAY` siguen siendo la defensa principal.

---

11. Cómo extenderlo

Idea	Dónde tocar
Citar con comentario en vez de repost	Tras abrir el menú, usar `[data-testid="quote"]`
Incluir respuestas	`X_SKIP_REPLIES=0`
Filtros con expresiones regulares	Cambiar `text_ok()` por `re.search` sobre `p["text"]`
Zona horaria configurable	Convertir `time` (UTC) a la zona local antes de comparar fechas
Notificar al terminar	Leer el historial o añadir un `curl` a un webhook al final de `main()`
Ejecución diaria	`cron` o `systemd --user` con `X_APPLY=1` y `--force` puntual
Versión con API oficial	OAuth 2.0 + `POST /2/users/:id/retweets`
Interfaz gráfica	Envolver `xrepost.sh` con `zenity` o una TUI

---

12. Verificación realizada

Todo lo siguiente se probó con la cuenta real, dejándola sin cambios (donde se indica, con un DOM simulado):

1. **Dry-run** con `kubernetesio` (27-oct-2026): 2 posts listados con URL correcta.
2. **Dry-run** con `kubernetesio` (20→28-oct-2026): 5 posts, incluyendo el salto de día correcto.
3. **Ciclo completo de repost** sobre un post público: `retweet` 0 → 1 tras confirmar, captura guardada.
4. **Deshacer**: `unretweet` 1 → 0, dejando la cuenta como estaba.
5. **Persistencia de sesión**: la cuenta detectada fue la del perfil de Chrome usado (`@rignack_X`).
6. **Modo interactivo**: con entrada simulada (`printf 'usuario\nfecha\n\n'`) respondió a los prompts y listó los posts correctos.
7. **Caso real con retweet mezclado**: buscando a `@garcia_yec52416` (21→28-oct-2026) la búsqueda devolvía 3 posts suyos y 1 retweet de otra cuenta; el filtro `X_SKIP_RETWEETS` descartó el retweet y dejó los 3 propios.
8. **Filtro por handle, no por texto**: ese retweet se descartó porque su autor (`@ManuelMilian9`) no coincide con el usuario buscado, sin depender de que la interfaz diga "reposted".
9. **Deduplicación global**: con `X_SKIP_RETWEETS=0 ./xrepost.sh garcia_yec52416,ManuelMilian9 2026-10-21 2026-10-28`, el post retuiteado apareció una sola vez pese a salir en las dos búsquedas.
10. **Respuestas por DOM**: buscando a `@ETECSA_Cuba` (25→28-oct-2026) se listaron solo los posts originales; las respuestas (una con tres menciones en la línea "Replying to") quedaron fuera

sin mirar el idioma.

11. **Ayuda**: `./xrepost.sh --help` muestra el bloque completo de variables y ya no imprime la línea `set -euo pipefail`.
12. **Filtros de texto y exportación** (dry-run real): `X_MATCH="zzzz-no-existe"` descartó los 2 posts y dejó el CSV solo con cabecera; sin filtro, `--csv` y `--md` guardaron los 2 posts con estado `solo listado`.
13. **Historial, idempotencia y deshacer** (DOM simulado, sin tocar la cuenta): repostar → repetir la ejecución (0 acciones) → `--undo-last` → `--undo` por ID → `--force`. Se comprobó que no duplica, que tras deshacer el post vuelve a estar disponible y que `--undo-last` no revierte un `ya estaba reposteado` ajeno.
14. **Modos sin navegador**: `--history` lee el JSONL y sale sin abrir Chrome; `--undo-last` sin candidatos tampoco navega.

Capturas de esas pruebas: `~/Downloads/demo-5-x-home.png`, `~/Downloads/demo-6-repost-test.png`.

---

13. Preguntas frecuentes

¿Necesito la API de X?
No. Se usa el navegador con tu sesión.

¿Necesito un modelo de IA o una clave de OpenAI/Anthropic?
No. Ver sección 3.1: el programa no llama a ningún modelo, solo controla Chrome por CDP.

¿Puedo repostear de varias cuentas a la vez?
Sí: `./xrepost.sh cuenta1,cuenta2,cuenta3 2026-10-27`.

¿Qué pasa si un post ya estaba reposteado?
Se detecta y se salta; no se duplica ni se deshace. Si ya consta en el historial, ni siquiera se vuelve a abrir.

¿Puedo deshacer un repost?
Sí: `./xrepost.sh --undo-last` deshace la última tanda y `./xrepost.sh --undo URL|ID`, uno concreto. Solo revierte reposts hechos por la herramienta, no los que ya estaban antes.

¿Dónde se guarda el historial? ¿Puedo borrarlo?
En `~/.local/state/xrepost/history.jsonl`. Bórralo (o cambia `X_HISTORY`) si quieres que la herramienta olvide lo que ya reposteó; no afecta a lo que hay en tu cuenta.

¿Cómo reposteo solo los posts que hablan de un tema?
Con `X_MATCH="kubernetes,cloud"` (debe contener alguna) y/o `X_EXCLUDE="empleo,oferta"` (descarta si contiene alguna).

¿Puedo repostear posts de fechas muy antiguas?
Sí, mientras la búsqueda de X los devuelva. X limita la profundidad del histórico; si no aparecen, no están indexados para búsqueda.

¿Funciona con cuentas protegidas?
Solo si tu sesión ya las sigue; si no, no aparecerán en la búsqueda.

¿Por qué no aparecen algunos de sus posts?
Los retweets y las respuestas se ignoran por defecto. Para incluirlos: `X_SKIP_RETWEETS=0` y/o `X_SKIP_REPLIES=0`.

¿Se puede programar?
Sí, con `cron` o `systemd --user`, siempre con `X_LIMIT` bajo.

¿Dónde está el límite de "demasiado"?
Cuando X empieza a mostrar `no se pudo confirmar` de forma repetida. Ahí hay que parar y bajar el ritmo.